

内部排序

在内存中怎样用局部进展制造全局有序

408 · 数据结构 Topic DS-11

母问题：每一步究竟把什么“已排序”变成事实？

内部排序的生成链是

Local Operation → Progress Measure → Ordering Invariant → Termination → Cost/Stability.

插入排序维护已排序前缀，选择排序维护已确定前缀，交换排序用 partition 缩小未决区间，归并排序把有序子段合并，堆排序借用 DS05 的根极值。算法区别不只在口诀，而在每轮不变量和数据搬运方式。

本册定位与 Owner 边界

- **Owns**：直接/折半插入、Shell、冒泡、简单选择、快速、堆、二路归并、基数排序，以及比较/移动、稳定性、原地性与最坏输入。
- **Uses**：DS05 堆的接口与 Atlas Foundation 成本语言。
- **Stops**：外部存储块 I/O 与多路归并归 DS12；选择哪种排序的 workload 规则进入 Rules。

目录

1	排序契约与三种性质	2
2	不变量驱动的四条主线	2
3	插入族：定位成本与移动成本必须分开	2
4	交换与选择族：每趟提交的事实不同	2
5	堆、二路归并与基数：改变候选组织方式	2
6	代码与测试证据	3
7	复杂度与概念边界	6
8	做题控制协议	6

1 排序契约与三种性质

排序先固定升序/降序、是否允许重复、是否要求稳定以及是否允许额外空间。稳定性保持相等 key 的原始相对次序；原地性描述额外空间，不等于稳定；比较排序的下界适用于只通过比较获得次序的算法，基数排序改变了信息模型。

2 不变量驱动的四条主线

插入排序：前缀有序，取下一个元素向左移动直到放置点；最好 $O(n)$ 、最坏 $O(n^2)$ ，稳定。选择排序：前缀是全局最小的若干元素，交换次数少但通常不稳定。快速排序：partition 后枢轴落在最终位置，递归处理两侧；平均 $O(n \log n)$ 、最坏 $O(n^2)$ 。归并排序：两个有序段按双指针合并，稳定且 $O(n \log n)$ ，需要线性辅助空间。

为什么“有序区”不是一句空话

对数组 $[4, 1, 3, 2]$ 做插入排序。处理完 4, 1 后前缀成为 $[1, 4]$ ；处理 3 时只需在该前缀内找插入点。若每轮都重新排序整个数组，就失去该不变量带来的结构化进展。

3 插入族：定位成本与移动成本必须分开

直接插入从有序前缀末尾向左比较并移动，直到找到插入位置。折半插入只用二分缩短“找位置”的比较次数，但连续数组仍要移动插入点后的元素，所以总移动量和最坏时间仍为 $O(n^2)$ ；它不是把插入排序整体变成 $O(n \log n)$ 。

Shell 排序选择递减增量 $d_1 > d_2 > \dots > 1$ ，每一趟对所有相距 d 的子序列做插入排序。大增量先让元素跨远距离移动，最后 $d = 1$ 时数组已经较接近有序。其正确性最终由最后一趟普通插入保证；复杂度依赖增量序列，不能在未给序列时统一宣称精确阶。跨距移动可能改变相等 key 的相对次序，因此 Shell 通常不稳定。

4 交换与选择族：每趟提交的事实不同

冒泡排序比较相邻逆序对并交换；一趟从左到右可把当前最大元素提交到未排序区末端。若整趟没有交换即可提前停止，因此已排序输入最好为 $O(n)$ 。简单选择排序每趟在未排序区找最小元素，与边界位置交换；它只进行 $O(n)$ 次交换，但一次跨越交换可能越过相等 key，通常不稳定。

快速排序用 partition 把区间分成枢轴两侧，并让至少一个分割边界成为进展。正确性依赖 partition 循环不变量；复杂度依赖分割是否均衡。已排序、逆序或大量重复 key 在某些固定枢轴实现中会持续产生极端分割，使递归深度和时间退化到 $O(n^2)$ 。

5 堆、二路归并与基数：改变候选组织方式

堆排序先调用 DS05 建立最大堆，每轮把根极值与未排序区末端交换，缩小堆边界并对新根下滤。已排序后缀不断扩大，时间稳定为 $O(n \log n)$ 、额外空间 $O(1)$ ，但根与末端的远距离交换使其不稳定。

二路归并把两个已有序段合成一个有序段：比较两段当前首元素，取较小者并推进对应指针；某段耗尽后复制另一段剩余部分。若相等时优先取左段，可保持稳定。自底向上的归并把长度 $1, 2, 4, \dots$ 的段逐轮合并，无论初始次序如何都为 $O(n \log n)$ ，代价是 $O(n)$ 辅助数组。

基数排序不比较完整 key，而是按某一位的桶序反复稳定分配与收集。最低位优先从低位到高位，后

一趟必须稳定，才能保留前面低位已建立的次序。若有 d 位、基数为 r ，成本为 $O(d(n+r))$ ；它依赖 key 可分解为有限位，不受比较排序下界约束。

一张表不能替代机制

遇到排序过程题，先写本趟提交的不变量：插入是有序前缀，冒泡是末端极值，选择是前端极值，快速是分区边界，堆是根极值加堆边界，归并是有序段，基数是已处理位上的稳定次序。然后才数比较、移动、趟数和空间。

6 代码与测试证据

完整实现位于 code/ds11_internal_sort.hpp，测试位于 code/ds11_internal_sort_test.cpp。

Listing 1: 插入、归并与快速排序核心转移

```

10
11 inline void insertion_sort(std::vector<int>& values) {
12     for (std::size_t i = 1; i < values.size(); ++i) {
13         int value = values[i]; std::size_t j = i;
14         while (j > 0 && values[j - 1] > value) { values[j] = values[j - 1];
15             --j; }
16         values[j] = value;
17     }
18
19 inline void selection_sort(std::vector<int>& values) {
20     for (std::size_t i = 0; i < values.size(); ++i) {
21         std::size_t minimum = i;
22         for (std::size_t j = i + 1; j < values.size(); ++j) if (values[j] <
23             values[minimum]) minimum = j;
24         std::swap(values[i], values[minimum]);
25     }
26
27 inline void merge(std::vector<int>& values, std::vector<int>& buffer, std::
28     size_t left, std::size_t mid, std::size_t right) {
29     std::size_t i = left, j = mid, k = left;
30     while (i < mid && j < right) buffer[k++] = values[i] <= values[j] ?
31         values[i++] : values[j++];
32     while (i < mid) buffer[k++] = values[i++];
33     while (j < right) buffer[k++] = values[j++];
34     for (k = left; k < right; ++k) values[k] = buffer[k];
35 }
36 inline void merge_sort_impl(std::vector<int>& values, std::vector<int>&
37     buffer, std::size_t left, std::size_t right) {
38     if (right - left < 2) return;
39     const auto mid = left + (right - left) / 2;
40     merge_sort_impl(values, buffer, left, mid); merge_sort_impl(values,

```

```

    buffer, mid, right); merge(values, buffer, left, mid, right);
38 }
39 inline void merge_sort(std::vector<int>& values) { std::vector<int> buffer(
    values.size()); merge_sort_impl(values, buffer, 0, values.size()); }
40
41 inline std::size_t partition(std::vector<int>& values, std::size_t left,
    std::size_t right) {
42     const int pivot = values[right - 1]; std::size_t store = left;
43     for (std::size_t i = left; i + 1 < right; ++i) if (values[i] < pivot) {
44         std::swap(values[i], values[store]); ++store; }
45     std::swap(values[store], values[right - 1]); return store;
46 }
47 inline void quick_sort_impl(std::vector<int>& values, std::size_t left,
    std::size_t right) {
48     if (right - left < 2) return;
49     const auto pivot = partition(values, left, right); quick_sort_impl(
    values, left, pivot); quick_sort_impl(values, pivot + 1, right);
50 }
51 inline void quick_sort(std::vector<int>& values) { quick_sort_impl(values
    , 0, values.size()); }
52
53 inline void heap_sort(std::vector<int>& values) {
54     auto sift_down = [&values](std::size_t start, std::size_t end) {
55         std::size_t root = start;
56         while (2 * root + 1 < end) {
57             std::size_t child = 2 * root + 1;
58             if (child + 1 < end && values[child] < values[child + 1]) ++child;
59             if (values[root] >= values[child]) break;
60             std::swap(values[root], values[child]); root = child;
61         }
62     };
63     for (std::size_t start = values.size() / 2; start > 0; --start)
64         sift_down(start - 1, values.size());
65     for (std::size_t end = values.size(); end > 1; --end) { std::swap(
66         values[0], values[end - 1]); sift_down(0, end - 1); }
67 }
68
69 inline void radix_sort_nonnegative(std::vector<int>& values) {
70     if (std::any_of(values.begin(), values.end(), [](int value) { return
71         value < 0; })) throw std::invalid_argument("radix sort requires
72         nonnegative keys");
73     int maximum = 0; for (const int value : values) maximum = std::max(
74         maximum, value);
75     std::vector<int> buffer(values.size());
76     for (int place = 1; maximum / place > 0; place *= 10) {
77         std::vector<std::size_t> count(10, 0);
78         for (const int value : values) ++count[static_cast<std::size_t>((value

```

```

    / place) % 10)];
73     for (std::size_t digit = 1; digit < 10; ++digit) count[digit] +=
count[digit - 1];
74     for (std::size_t i = values.size(); i > 0; --i) { const int value =
values[i - 1]; const std::size_t digit = static_cast<std::size_t>((
value / place) % 10); buffer[--count[digit]] = value; }
75     values.swap(buffer);
76     if (place > maximum / 10) break;
77 }
78 }
79 } // namespace ipara::ds11
80
81 #endif

```

Listing 2: 空、重复、已排序与逆序边界测试

```

8     const std::vector<int> input{4, 1, 3, 2, 2};
9     for (auto sorter : {ipara::ds11::insertion_sort, ipara::ds11::
selection_sort, ipara::ds11::merge_sort, ipara::ds11::quick_sort,
ipara::ds11::heap_sort}) {
10         auto values = input; sorter(values); assert((values == std::vector<
int>{1, 2, 2, 3, 4}));
11     }
12     std::vector<int> empty; ipara::ds11::merge_sort(empty); assert(empty.
empty());
13     std::vector<int> sorted{1, 2, 3}; ipara::ds11::quick_sort(sorted);
assert((sorted == std::vector<int>{1, 2, 3}));
14     std::vector<int> nonnegative{170, 45, 75, 90, 802, 24, 2, 66}; ipara::
ds11::radix_sort_nonnegative(nonnegative); assert((nonnegative == std
::vector<int>{2, 24, 45, 66, 75, 90, 170, 802}));
15     bool negative = false; std::vector<int> with_negative{1, -1}; try {
ipara::ds11::radix_sort_nonnegative(with_negative); } catch (const std
::invalid_argument&) { negative = true; } assert(negative);
16 }

```

```

clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/
    ds11_internal_sort_test.cpp -o /tmp/ds11_test
/tmp/ds11_test

```

7 复杂度与概念边界

算法	最好	平均	最坏	稳定/空间
直接/折半插入	n	n^2	n^2	稳定、原地
冒泡	n	n^2	n^2	稳定、原地
简单选择	n^2	n^2	n^2	不稳定、原地
Shell	依增量	依增量	依增量	不稳定、原地
归并	$n \log n$	$n \log n$	$n \log n$	稳定、 $O(n)$
快速	$n \log n$	$n \log n$	n^2	通常不稳定
堆	$n \log n$	$n \log n$	$n \log n$	不稳定、原地
基数	$d(n + r)$	$d(n + r)$	$d(n + r)$	稳定分配、 $O(n + r)$

8 做题控制协议

先写“哪一段已排序/哪个枢轴已归位”，再数比较、移动和辅助空间。看到稳定性要求优先排除会跨越相等 key 的交换；看到最坏保证与线性空间权衡，比较归并、堆与快速的不同前提。DS05 只提供堆操作，堆排序的整体不变量由本册负责。

压缩复原

五问：已确定区域在哪里？一次局部操作扩大了什么？终止量是什么？相等 key 的次序是否保留？最坏输入会让哪一支退化？